

Day35 · LangChain Document Loaders (RAG 第 1 步)

环境: 虚拟环境 .venv (langchain_lab) , LangChain v1 (1.4.2) + langchain_community
模型: 暂不需要 (今天不调模型, 只加载文档)

一、先串一遍: RAG 是什么 (第 8 周主题)

RAG = 检索增强生成 (Retrieval-Augmented Generation) —— 让大模型"先查资料再回答", 解决"模型只会背知识点、不懂你的私有文档"的问题。

RAG 全流程 (6 步, 第 8 周会一步步做) :

① 文档加载 (今天) → ② 文本分割 (Day36) → ③ 向量化 (Day37) → ④ 存储 (Day37) → ⑤ 检索 (Day38) → ⑥ 生成回答 (拼进 prompt)

对照你学过的:

平台/之前学的	本质
Day25 Coze 知识库 / Day26 客服	平台帮你做了"上传文档→切分→检索" (自动 RAG)
Day22-24 手写 RAG / 文档问答	你自己手动实现了"切分+向量+检索"
今天 LangChain	用框架标准组件来做——换一个说法: Day22-24 的手写代码, LangChain 封装成了 <code>Document Loader</code>

一句话: RAG 的 6 步里, 今天只做第 ① 步: 把各种格式 (PDF/CSV/网页/Word...) 的原始文件, 变成 LangChain 统一的 `Document` 对象。

二、Document Loader 是什么 (核心概念)

一句话: Document Loader (文档加载器) 是 LangChain 的"万能读卡器"——把不同类型的外部数据 (PDF、CSV、网页、Word、数据库...) 读进来, 统一成一种标准结构 `Document` 。

```
python
```

```
from langchain_core.documents import Document
# 每个 Document 包含两部分：
doc.page_content    # 正文文本
doc.metadata        # 元信息（来源路径、页码、行号...）
```

为什么要统一？因为后续 RAG 的 ② 分割 ③ 向量化 ⑤ 检索 都只认 `Document` 这一种结构——**Loader** 负责“万物归一”，后面的步骤就不用管来源是 PDF 还是网页了。这就是框架的价值：接口统一、可插拔。

所有 Loader 的通用接口（背）：

```
python
```

```
docs = loader.load()          # 一次加载全部，返回 [Document, Document, ...]
docs = loader.lazy_load()     # 惰性加载，逐条产出，适合大文件省内存
```

三、开始前：装依赖（终端执行）

```
bash
```

```
pip install langchain-community pypdf beautifulsoup4 requests
```

- `langchain-community`：官方社区包，Document Loader 大多在这
- `pypdf`：PDF 解析库（PyPDFLoader 依赖）
- `beautifulsoup4`：网页解析库（WebBaseLoader 依赖）
- `requests`：抓网页

四、练习① 加载 PDF 文档

先准备一个 PDF。最简单的办法：把之前的学习计划或任意一篇记的东西，用记事本存成 `.txt` 再改成 `.pdf` 不行（PDF 有格式），所以请你用一个真正的 PDF，比如：

- 从网上下载任意一份公开 PDF
- 或者用浏览器把本指南打印成 PDF（Ctrl+P → 另存为 PDF）放桌面

把 PDF 放到 `D:\python_learning\langchain_lab` 下，命名为 `test.pdf`。

新建 `51_pdf_loader.py` :

python

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("test.pdf")          # 指向你的 PDF 文件
docs = loader.load()                      # 加载全部，返回页面列表

print(f"共加载了 {len(docs)} 页")
print("=== 第1页内容 (前300字) ===")
print(docs[0].page_content[:300])
print("=== 第1页元信息 ===")
print(docs[0].metadata)
```

跑通标准：打印出 `共加载了 N 页`、第 1 页开头文字、以及 `metadata`（里面应有 `source: test.pdf` 和 `page: 0`）。

概念点（背）：

- `PyPDFLoader` 按页切分：一页 → 一个 `Document`（`mode` 参数可设 `"single"` 整篇一个）。
- `docs[0].page_content` 取第 1 页正文，`docs[0].metadata` 取第 1 页元信息。

五、练习② 加载 CSV 数据

在 `langchain_lab` 下新建 `数据.csv`（文件名不要用中文，避免编码坑）：

CSV

```
产品,价格,库存
机械键盘,299,50
电竞鼠标,159,120
显示器,1299,30
音箱,399,80
```

新建 `52_csv_loader.py` :

```
from langchain_community.document_loaders import CSVLoader

loader = CSVLoader("数据.csv")  # CSVLoader 会自动处理表头
```

```
python
docs = loader.load()

print(f"共加载了 {len(docs)} 行")
for i, doc in enumerate(docs[:3]):      # 只看前3行
    print(f"--- 第{i}行 ---")
    print(doc.page_content)
    print("元信息:", doc.metadata)
```

跑通标准：共加载了 4 行，每一行 `page_content` 是 产品：机械键盘\n价格：299\n库存：50 这样的键值对文本，`metadata` 里有 `row: 0/1/2...`。

概念点：

- CSV 特点：每一行 → 一个 Document（因为 CSV 是表格，一行就是一条记录）。
- `source_column` 参数可指定某一列作为元信息来源：

```
python

loader = CSVLoader("数据.csv", source_column="产品")
# 这样 source 就变成"机械键盘"而不是文件路径，方便回答时引用来源
```

- `csv_args` 参数可传自定义分隔符（如制表符 `"\t"`）、指定列名等。

六、练习③ 加载网页内容

新建 `53_web_loader.py`：

```
python

from langchain_community.document_loaders import WebBaseLoader

url = "https://www.python.org/about/"      # 一个稳定的公开网页
loader = WebBaseLoader(url)
docs = loader.load()

print(f"加载了 {len(docs)} 个文档块")
print("=== 网页正文（前500字）===")
print(docs[0].page_content[:500])
print("=== 元信息 ===")
print(docs[0].metadata)
```

跑通标准：打印出 Python 官网关于页的正文前 500 字、`metadata`（含 `source: 网址`、`title`）。

注意： `WebBaseLoader` 依赖 `beautifulsoup4`，如果没装会报 `ModuleNotFoundError`。若网络慢或网页反爬，可换一个新闻/文档静态网页试。

概念点：

- 网页 Loader 会自动剥掉 HTML 标签，只留正文文本（这正是给 LLM 看的格式）。
- metadata 里会带 `source`（网址）、`title`（标题）——检索后能注明引用来源（RAG 展示出处就靠它）。

七、练习④ 了解不同 Loader 的适用场景（不需要写代码，看完背下来）

Loader	来源	输出特点	适用场景
<code>TextLoader</code>	.txt 纯文本	整个文件一个 Document	纯文本笔记、日志
<code>PyPDFLoader</code>	.pdf	按页切分，一页一个 Document	论文、合同、书籍、说明书
<code>CSVLoader</code>	.csv 表格	一行一个 Document	结构化数据表、excel 导出
<code>WebBaseLoader</code>	网页 URL	剥 HTML 只留正文	官网、文档站、新闻抓取
<code>Docx2txtLoader</code>	.docx	整个文档	Word 文档
<code>DirectoryLoader</code>	整个文件夹	批量加载文件夹内所有文件	一次性喂一批资料

记忆技巧：Loader 的共通点就 3 个——加载什么格式、按什么粒度切（页/行/整篇）、metadata 带什么。面试时能说清这 3 点+举 2 个例子就够。

八、今晚整理：Document Loader 笔记要点

1. RAG 6 步流程，今天做第 1 步"文档加载"
2. Loader 把万物统一成 Document（page_content + metadata），后续分割/向量化只认这个结构
3. 通用接口：`load()` 全量 / `lazy_load()` 惰性

4. 3 个动手练习的加载命令 + 切换要点（PDF按页、CSV按行、网页剥标签）

5. 一张 Loader 适用场景表

面试话术（Day35 可讲的点）

Q：在用 LangChain 做 RAG 吗？介绍一下加载文档这块。

A：RAG 的第一步是文档加载，LangChain 的 Document Loader 负责把我各种格式的数据统一读成标准的 Document 对象（正文 + 元信息）。我实际加载过 PDF（按页切，带页码元信息）、CSV（按行，每行一个文档、能指定来源列）、网页（自动剥掉 HTML 标签只留正文、带网址和标题）。这样后续的文本分割、向量化就能用统一接口处理，不用管数据原始是什么格式。

本内容由 Coze AI 生成，请遵循相关法律法规及《人工智能生成合成内容标识办法》使用与传播。